Smart Academic and Examination Management System (SAEMS)

Software Requirement Engineering (Lab)



Submitted by: Abdul Samad

Submitted to: Sir Fahim Muhammad Khan

Lab No. 5

Semester: V - BSSE (Group-A)

Use Case Modeling

Session 2024 –2028 (Fall)

INSTITUTE OF MANAGEMENT SCIENCES

HAYATABAD, PESHAWAR (2025)

1. Task 1 - Identify the System Boundary

System name: Smart Academic and Examination Management System (SAEMS).

Inside the boundary: The SAEMS use cases that support academic and examination activities, including course registration, course dropping, timetable and examination schedule viewing, instructor work, coordinator work, examination work, and administrator functions.

Outside the boundary: The actors that interact with SAEMS. The scenario specifically mentions the Identity Authentication Service and Bank Payment Gateway as external systems. The human actors are also outside the boundary.

2. Task 2 - Identify Actors

Actor	Type	Reason for Interaction
Student	Primary, human	Uses SAEMS for course search, registration, dropping courses, timetable and examination schedule.
Course Instructor	Primary, human	Uses SAEMS to view assigned courses and students, upload materials, submit attendance and enter marks.
Department Coordinator	Primary, human	Manages course offerings, instructor assignments and special registration/drop requests.
Examination Officer	Primary, human	Creates examination schedules, assigns rooms, publishes the timetable and records results.
System Administrator	Primary, human	Manages accounts, roles, permissions, configuration and system reports.
Identity Authentication Service	Secondary, external system	Used when a user logs in and when password recovery is needed.
Bank Payment Gateway	Secondary, external system	Processes the student's outstanding dues payment and returns the payment result.

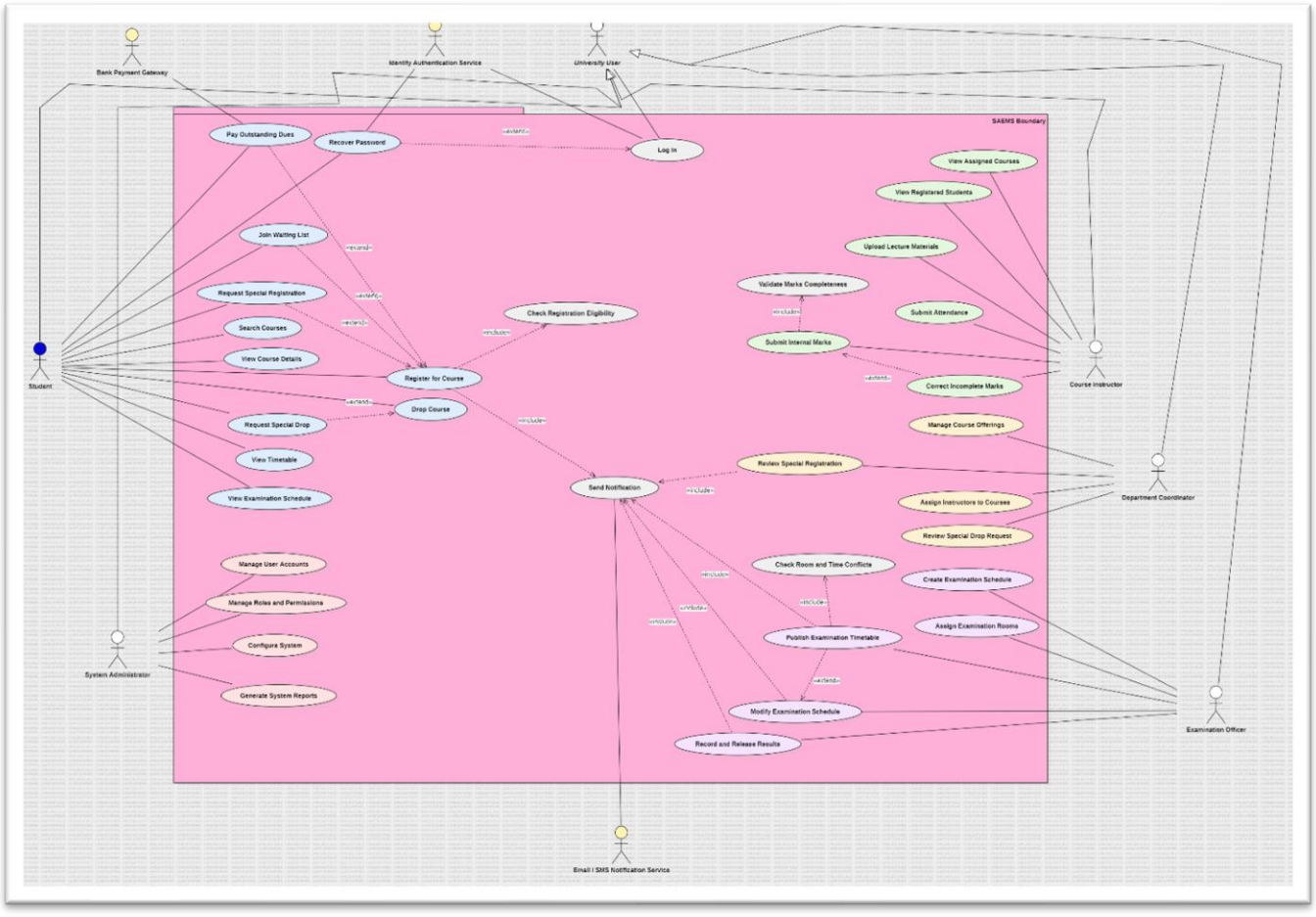
3. Task 3 - Identify Actor Goals / Candidate Use Cases

Actor	Candidate Goal / Use Case
Student	Search Courses View Course Details Register for Course Drop Course View Timetable View Examination Schedule

Course Instructor	View Assigned Courses View Registered Students Upload Lecture Materials Submit Attendance Enter Internal Marks
Department Coordinator	Manage Course Offerings Assign Instructors to Courses Review Special Registration Review Special Drop Request
Examination Officer	Create Examination Schedule Assign Examination Rooms Publish Examination Timetable Record and Release Results
Actor	**Candidate Goal / Use Case**
System Administrator	Manage User Accounts Manage Roles and Permissions Configure System Generate System Reports
University User (all five human roles)	Log In

4. Task 4 - Draw the Basic Use Case Diagram

The supplied StarUML model is included below as the diagram used for this report.



5. Task 5 - Analyze «include» and «extend»

The relationships below follow the completed model. «include» is used for behavior that is required as part of the base use case. «extend» is used where the extra behavior happens only in a particular condition.

Extension	Base	Condition
Recover Password	Log In	Student forgot password
Join Waiting List	Register for Course	Course is full
Pay Outstanding Dues	Register for Course	Student has dues
Request Special Registration	Register for Course	Prerequisites missing
Submit Special Drop Request	Drop Course	Deadline has passed
Modify Examination Schedule	Publish Examination Timetable	Room or time conflict found

Base	Included
Register for Course	Check Registration Eligibility (prerequisites and seats)
Enter Internal Marks	Validate Marks Completeness
Publish Examination Timetable	Check Schedule Conflicts

These checks are kept as separate included use cases in the model. They could also be written as steps inside the fully dressed use case, but the important point is to keep the chosen approach consistent.

6. Task 6 - Consider Generalization

An abstract actor called University User is used for the five human roles. Student, Course Instructor, Department Coordinator, Examination Officer and System Administrator inherit the shared Log In use case from this general actor.

This also handles the case where one person performs more than one university role: the person can act through the relevant specialized actor roles.

7. Task 7 - Review the Model Like a Requirements Analyst

System boundary is clearly defined.

Actors are outside the system boundary.

Use cases are inside the system boundary.

Use case names describe meaningful goals.

Each actor has a reason for interacting with SAEMS.

User-interface actions are not used as separate use cases.

«include» relationships are used for required checks.

«extend» relationships are used for conditional or optional behavior.

Relationship directions and labels are shown in the diagram.

The diagram is kept as the supplied model rather than adding unrelated features.

8. Task 8 - Write One Use Case Description

Use Case Name	Register for Course
Primary Actor	Student
Goal	Register for a selected course successfully through SAEMS.
Preconditions	The student is logged in and has selected a course to register for.
Main Success Scenario	1. The student starts the course registration. 2. SAEMS checks the student's identity, prerequisites and available seats. 3. The registration is completed when the required checks are satisfied. 4. SAEMS generates a registration confirmation. 5. SAEMS updates the student's timetable.
Alternative / Exception Flows	If the course is full, the student may join the waiting list. If the student has outstanding dues, the dues are paid through the Bank Payment Gateway before registration is completed. If prerequisites are missing, the student may submit a special registration request.
Postconditions	The student is registered for the course and the timetable is updated.

9. Lab Reflection Questions

1. What is the difference between an actor and a user?

An actor is an external entity that interacts with the system to achieve a goal. A user is a person who uses the system. In a use case model, an actor can also be another system or an external service.

2. Why must actors normally be outside the system boundary?

The boundary shows what belongs to the system being modeled. Actors are outside because they interact with the system rather than being part of the system itself.

3. Why is "Register for Course" a stronger use case candidate than "Click Register Button"?

"Register for Course" describes a meaningful goal the student wants to complete. "Click Register Button" is only a user-interface action.

4. What is the practical difference between «include» and «extend»?

«include» is used when the included behavior is required as part of the base use case. «extend» is used when extra behavior happens only under a condition or is optional.

5. What mistake would occur if every system function were treated as a separate use case?

The model would contain too many small and unnecessary use cases. It would become harder to read and would mix internal system activities with meaningful user goals.

6. How does a Use Case Diagram differ from a fully dressed Use Case?

A use case diagram shows actors, use cases and their relationships at a high level. A fully dressed use case gives more detail about one use case, such as its actor, goal, preconditions, main steps, alternatives and postconditions.

7. What information does a Use Case Diagram intentionally leave out?

It leaves out the detailed step-by-step flow, alternative flows, preconditions and postconditions of each use case. Those details are written in the fully dressed use case.

Key Takeaway

Use case modeling focuses on who interacts with the system, what meaningful goals they want to achieve, and how those interactions are represented at a high level.